

Subject

Speech and Natural Language Processing

Professor

Isaac Gonzalez

Session 09

Unit II

Deep learning for NLP

Topic 2.4

Fine-Tuning Pre-trained Language Models

Fine-Tuning

The Pre-train + Fine-tune Paradigm

- Introduction

The Paradigm Shift in NLP

Traditional NLP: train task-specific models from scratch

Problem: requires large labeled datasets for each task

New paradigm: Pre-train + Fine-tune

Pre-train a large model on unlabeled text (self-supervised)

Fine-tune the pre-trained model on a small labeled dataset

This is Transfer Learning applied to NLP

Why Transfer Learning Works

Language has shared structure across tasks

Pre-trained models learn syntax, semantics, world knowledge

Lower layers: general features (word structure, syntax)

Higher layers: task-specific features

Similar to computer vision: ImageNet pre-training

Dramatically reduces labeled data requirements

Two Approaches to Transfer Learning

Feature-based (e.g., ELMo):

- Use pre-trained representations as input features

- Train a new classifier on top (frozen representations)

Fine-tuning (e.g., BERT, GPT):

- Initialize model with pre-trained weights

- Update ALL parameters on the downstream task

Fine-Tuning

From Static to Dynamic Representations

- Contextualized Embeddings

From Static to Contextual Embeddings

Word2Vec / GloVe: one vector per word type

Problem: 'bank' has same vector in all contexts

Contextual embeddings: different vector for each use

The representation depends on the surrounding words

Same word in different sentences → different vectors

ELMo: Embeddings from Language Models

Peters et al. (2018)

Train a bidirectional LSTM language model

Forward LM: predict next word (left-to-right)

Backward LM: predict previous word (right-to-left)

Combine hidden states from ALL layers

Final embedding = weighted sum of all layer outputs

Weights are learned per downstream task

Using ELMo in Practice

Feature-based approach:

- Freeze ELMo weights
- Concatenate ELMo vectors with task input embeddings
- Train task-specific layers on top

Different layers capture different information:

Layer 1 (syntax): POS tagging, dependency parsing

Layer 2 (semantics): word sense, sentiment

Fine-Tuning

Bidirectional Encoder Representations

- BERT

BERT: Overview

Devlin et al. (2018) - Google

Key innovation: bidirectional pre-training

Uses Transformer encoder (architecture details in S10-S11)

Pre-trained on large unlabeled text corpora

Two pre-training objectives: MLM and NSP

State-of-the-art on 11 NLP tasks when released

BERT Pre-training: Masked Language Model (MLM)

Randomly mask 15% of input tokens

Model must predict the masked tokens

- Input: "The [MASK] sat on the mat"
- Target: "The cat sat on the mat"

This forces bidirectional context understanding

Of the 15% selected: 80% [MASK], 10% random, 10% kept

BERT Pre-training: Next Sentence Prediction

Binary classification: does sentence B follow A?

- 50% of the time B follows A (IsNext)
- 50% of the time B is random (NotNext)

Helps model understand sentence relationships

Useful for tasks like question answering, NLI

Later work (RoBERTa) showed NSP may not be necessary

BERT Input Representation

Three types of embeddings summed together:

- Token embeddings: WordPiece tokenization
- Segment embeddings: sentence A vs sentence B
- Position embeddings: learned, up to 512 tokens

Special tokens:

- [CLS]: classification token (first position)
- [SEP]: separator between sentences

BERT Architecture Details

BERT-Base: 12 layers, 768 hidden, 12 heads, 110M params

BERT-Large: 24 layers, 1024 hidden, 16 heads, 340M params

Pre-trained on BooksCorpus + Wikipedia (3.3B words total)

Training: 4-16 TPUs for 4 days

Transformer architecture details in Sessions 10-11

Fine-Tuning

Adapting BERT to Downstream Tasks

- Fine-Tuning BERT

Fine-Tuning: The General Framework

Initialize with pre-trained BERT weights

Add a task-specific output layer on top

Fine-tune ALL parameters end-to-end

Typically 2-4 epochs of training

Much less data needed than training from scratch

Learning rate: $2e-5$ to $5e-5$ (smaller than pre-training)

Task 1: Text Classification

Sentiment analysis, topic classification, spam detection

Use [CLS] token representation as sentence embedding

Add a linear classifier: $W * h_{[CLS]} + b$

Fine-tune entire model + classifier

Can handle single sentence or sentence pair input

Task 2: Token Classification (NER, POS)

Named Entity Recognition, POS Tagging

Use the representation of EACH token

Add a classifier on top of each token's output

Predict label for each token independently

WordPiece tokens: only classify first sub-token of each word

Task 3: Question Answering

Input: question + passage (context)

Output: start and end positions of the answer span

Two vectors learned: start vector S , end vector E

$P(\text{token } i \text{ is start}) = \text{softmax}(S \cdot h_i)$

$P(\text{token } j \text{ is end}) = \text{softmax}(E \cdot h_j)$

Answer = span with highest $P(\text{start}) \times P(\text{end})$

Task 4: Natural Language Inference (NLI)

Input: premise + hypothesis (sentence pair)

Output: relationship between sentences

- Entailment, Contradiction, or Neutral

Use [CLS] representation for classification

Segment embeddings distinguish premise vs hypothesis

Example benchmarks: MNLI, SNLI, QNLI

Fine-Tuning

Generative Pre-trained Transformer

- GPT

GPT: Overview

Radford et al. (2018) - OpenAI

Autoregressive language model (left-to-right only)

Uses Transformer decoder (architecture details in S10-S11)

Pre-trained to predict next token

Fine-tuned with task-specific heads

GPT-1: 117M parameters, BooksCorpus

GPT vs. BERT: Key Differences

BERT: bidirectional (sees full context)

GPT: unidirectional (sees only left context)

BERT: encoder-only, better for understanding tasks

GPT: decoder-only, better for generation tasks

BERT uses [CLS] token for classification

GPT uses last token representation for classification

The Scaling Hypothesis

GPT-2 (2019): 1.5B parameters, WebText dataset

GPT-3 (2020): 175B parameters, zero/few-shot learning

Key insight: scale → emergent abilities

In-context learning: no fine-tuning, just prompts

Few-shot: provide examples in the prompt

Led to the current era of large language models (LLMs)

Fine-Tuning

Techniques and Best Practices

- Practical Fine-Tuning

Hyperparameters for Fine-Tuning

Learning rate: $1e-5$ to $5e-5$ (critical choice)

Batch size: 16 or 32 (larger if GPU memory allows)

Epochs: 2-4 (more risks overfitting)

Warmup steps: 6-10% of total steps

Weight decay: 0.01 (regularization)

Max sequence length: task-dependent (128, 256, 512)

Catastrophic Forgetting

Problem: fine-tuning can overwrite pre-trained knowledge

Model forgets general language understanding

Solutions:

- Use a small learning rate (preserves pre-trained weights)
- Gradual unfreezing: fine-tune top layers first
- Regularization: penalize large weight changes
- Early stopping: don't fine-tune for too many epochs

Data Augmentation for Fine-Tuning

When labeled data is scarce:

Augmentation strategies:

- Synonym replacement: replace words with synonyms
- Back-translation: translate to another language and back
- Random insertion/deletion/swap of words
- Paraphrasing with another model

Even small augmentation can help significantly

Discriminative Fine-Tuning

Howard & Ruder (2018) - ULMFiT

Different learning rates for different layers

Lower layers: smaller learning rate (general features)

Higher layers: larger learning rate (task-specific)

Slanted triangular LR: brief warmup then linear decay

Gradual unfreezing: unfreeze one layer at a time

Fine-Tuning

PEFT Methods

- Parameter-Efficient Fine-Tuning

The Problem with Full Fine-Tuning

Full fine-tuning updates ALL model parameters

For a 7B model: need to store 7B updated parameters

Multiple tasks = multiple full copies of the model

Expensive in compute and storage

Solution: only update a small subset of parameters

Parameter-Efficient Fine-Tuning (PEFT) methods

Adapter Layers

Houlsby et al. (2019)

Insert small trainable modules between frozen layers

Adapter: Linear down-projection $\rightarrow$ ReLU $\rightarrow$ Linear up-projection

Bottleneck architecture: compress then expand

Only train adapter parameters ($\sim 2\text{-}4\%$ of total)

Original model weights stay frozen

LoRA: Low-Rank Adaptation

Hu et al. (2022) - Most popular PEFT method

Key idea: weight updates have low intrinsic rank

Instead of updating W directly:

- $W' = W + \Delta W = W + B * A$
- $A: (d \times r)$, $B: (r \times d)$, where $r \ll d$

Freeze W , only train A and B (much fewer params)

Achieves comparable performance to full fine-tuning

Prompt Tuning & Prefix Tuning

Prompt Tuning (Lester et al., 2021):

- Add learnable 'soft prompts' to the input
- Only train the prompt embeddings (~0.01% of params)

Prefix Tuning (Li & Liang, 2021):

- Add learnable prefix vectors to each layer
- More expressive than prompt tuning

Both: freeze the entire model, only train prompts/prefixes

Comparison of PEFT Methods

Full fine-tuning: best performance, highest cost

LoRA: near full-FT performance, ~1% params, most popular

Adapters: good performance, ~2-4% params, extra latency

Prompt Tuning: lowest cost, works well for large models

QLoRA: LoRA + 4-bit quantization, very memory efficient

Choose based on compute budget and model size

Fine-Tuning

Ollama, LM Studio, and GGUF

- Running LLMs Locally

Why Run LLMs Locally?

Privacy: sensitive data never leaves your machine

Cost: no per-token API fees after hardware investment

Latency: no network round-trips, instant responses

Offline access: works without internet connection

Customization: fine-tune and modify models freely

Education: understand what's really happening inside

The GGUF Model Format

GGUF = GPT-Generated Unified Format

Standard format for local LLM inference (llama.cpp)

Contains model weights + tokenizer + metadata in one file

Supports quantization for reduced memory usage

Used by Ollama, LM Studio, and other local tools

Available on Hugging Face for most open models

Quantization for Local Inference

Full precision (FP32): 4 bytes per parameter

Half precision (FP16): 2 bytes per parameter

Quantized (Q4): ~0.5 bytes per parameter

Rule of thumb: ~0.6-0.7 GB VRAM per billion params (Q4)

Common quantization levels:

Q4_K_M: best balance of size vs. quality (recommended)

Q5_K_M: slightly better quality, more memory

Q8_0: near-original quality, 2x memory of Q4

Ollama: CLI-First LLM Runtime

Open-source, 100K+ GitHub stars

Simple CLI: `ollama run llama3.2`

Automatic model download and management

OpenAI-compatible REST API (`localhost:11434`)

Modelfile: configure system prompt, parameters, template

Supports: Llama, Mistral, Gemma, Phi, Qwen, and more

Ollama: Key Commands

`ollama pull <model>` — download a model

`ollama run <model>` — start interactive chat

`ollama list` — show downloaded models

`ollama show <model>` — model details and parameters

`ollama create <name> -f Modelfile` — create custom model

`ollama serve` — start the API server

Ollama Modelfile

Recipe for custom models (like Dockerfile):

```
FROM llama3.2
```

```
PARAMETER temperature 0.7
```

```
PARAMETER num_ctx 4096
```

```
SYSTEM "You are a helpful NLP assistant."
```

Create: `ollama create my-model -f Modelfile`

Run: `ollama run my-model`

LM Studio: Visual LLM Desktop App

GUI-based desktop application (Win, macOS, Linux)

Browse and download models from Hugging Face

Visual VRAM usage estimates before loading

Built-in chat playground with conversation history

Side-by-side model comparison and benchmarking

Local API server (OpenAI-compatible)

Ollama vs. LM Studio

Ollama: CLI-first, better for developers and automation

LM Studio: GUI-first, better for experimentation

Ollama: faster inference (~20% in benchmarks)

LM Studio: Vulkan support (better without NVIDIA GPU)

Ollama: Modelfile for custom configurations

LM Studio: visual model management and comparison

Both: GGUF format, local OpenAI-compatible API

From Fine-Tuning to Local Deployment

1. Fine-tune with LoRA/QLoRA (e.g., using Unsloth)
2. Merge adapter with base model
3. Convert to GGUF format (llama.cpp convert script)
4. Quantize to desired precision (Q4_K_M, Q5_K_M)
5. Create Ollama Modelfile pointing to GGUF file
6. `ollama create my-model -f Modelfile` → ready to use!

Hardware Requirements for Local LLMs

VRAM is the key bottleneck for local LLMs

7B model (Q4_K_M): ~4-5 GB VRAM → RTX 3060 (12GB)

13B model (Q4_K_M): ~8-9 GB VRAM → RTX 4070 (12GB)

70B model (Q4_K_M): ~40 GB VRAM → multiple GPUs

Apple Silicon: unified memory (M1 Pro+: 16-64GB)

CPU-only inference possible but much slower

Fine-Tuning

Instruction Tuning, RLHF, and Summary

- Modern Trends

Instruction Tuning

Fine-tune on diverse tasks formatted as instructions

Examples:

- 'Summarize the following text: ...'
- 'Translate this sentence to Spanish: ...'

Models learn to follow instructions generally

Key datasets: FLAN, InstructGPT training data

Results in much more useful and controllable models

RLHF: RL from Human Feedback

Step 1: Supervised fine-tuning on demonstrations

Step 2: Train a reward model from human preferences

Step 3: Optimize policy with PPO against reward model

Used in ChatGPT, Claude, and other assistants

Aligns model behavior with human preferences

DPO (Direct Preference Optimization): simpler alternative

Choosing the Right Approach

Choose your approach based on:

- Resources available (compute, memory, time)
- Task requirements (generation vs. classification)
- Model size (full fine-tuning feasible for $< 1\text{B}$)

Small model + simple task → Full fine-tuning

Large model + limited compute → LoRA / QLoRA

Very large model + minimal resources → Prompt tuning

Just need to use a model → Ollama / LM Studio

Summary

Pre-train + Fine-tune: the modern NLP paradigm

ELMo: contextual embeddings via bidirectional LSTM

BERT: bidirectional pre-training (MLM + NSP)

GPT: autoregressive generation, scaling hypothesis

PEFT: LoRA, adapters, prompt tuning for efficiency

Local LLMs: Ollama (CLI) and LM Studio (GUI)

Next: Transformer architecture in detail (S10-S11)

Let's do:

S09_1_Exercises